

BEGINNER FRIENDLY GUIDE

Inside CareerHub The `src` Folder Explained

How the Front-End is built — folder by folder, code by code, with pictures of what each part draws on screen and how it talks to a C# back-end.

Next.js 15 · React 19 · TypeScript · Tailwind CSS

Chapters 0–7 · Two Summaries · 40 Interview Questions & Answers

With real screenshots captured from the running app · 2026

Table of Contents

- **Chapter 0** — `src` : the parent folder (the whole front-end lives here)
- **Chapter 1** — `app` : the pages & routes the user actually visits
- **Chapter 2** — `components` : the reusable building blocks of the screen
- **Chapter 3** — `context` : shared memory (who is logged in, pop-up messages)
- **Chapter 4** — `fonts` : the letters and typefaces
- **Chapter 5** — `lib` : the "engine room" — helpers, data & formatting
- **Chapter 6** — `types` : the agreed shape of the data (the contract)
- **Chapter 7** — Front-End connecting to a C# Back-End
- **Summary — Part 1** (the front-end folders, Chapters 0–6)
- **Summary — Part 2** (connecting to the back-end, Chapter 7)
- **Interview Questions & Answers** — 5 per chapter (40 total)

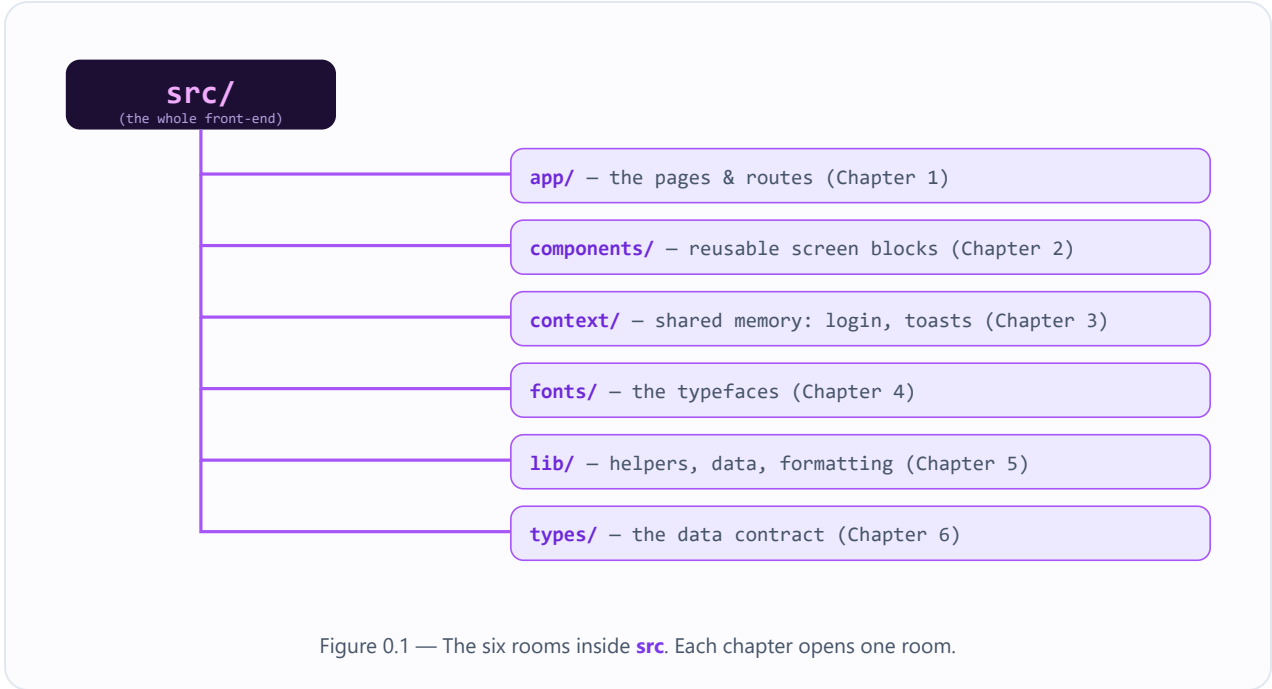
How to read this guide. You do not need to be a programmer. Think of the website as a house. The `src` folder is the whole house. Each sub-folder is a different *room* with a job to do. We will walk room by room, look at the real code that lives there, and — most importantly — show a picture of what that code **draws on the screen** and name the exact file responsible for it.

One word you will see a lot: "Front-End". The Front-End (also called the UI, or User Interface) is everything the user sees and clicks in their browser — buttons, text, colours, cards. The **Back-End** is the hidden computer (the server) that stores the real data. CareerHub's back-end is written in **C#**; Chapter 7 explains how the two talk.

The `src` Folder — the whole front-end

`src` is short for **source** — the source code of the app. Everything a developer writes lives inside it. When the app is built, the tools read `src` and turn it into the website you see in the browser.

The map of the house



Why split it into folders at all?

Imagine one giant file with every button, every colour and every rule mixed together. Changing one thing would risk breaking everything. Folders keep **related things together** and **unrelated things apart**. A new developer can guess where something lives: a page? look in `app`. A button? `components`. The colour of a salary label? `lib/format.ts`.

Folder	Plain-English job	Real example from CareerHub
<code>app</code>	The actual web pages people open	The home page listing jobs
<code>components</code>	Pieces reused on many pages	The job card, the navbar
<code>context</code>	Information shared everywhere	"Who is logged in?"
<code>fonts</code>	The letter shapes	Inter & Plus Jakarta Sans
<code>lib</code>	Behind-the-scenes logic	Turning <code>45000</code> into <code>R45 000</code>
<code>types</code>	The shape data must follow	What a "Job" must contain

Takeaway: `src` is not code by itself — it is the *container*. Its real value is the organisation. Good organisation is what lets a team build a large app without chaos.

app — the pages the user visits

In Next.js, the `app` folder is special: **every folder inside it becomes a web address (a URL)**, and a file named `page.tsx` is the screen shown at that address. This is called **file-based routing** — the folder structure is the site map.

The site map, built from folders

Folder / file in <code>app</code>	Web address	What the user sees
<code>page.tsx</code>	<code>/</code>	Home — the job listings
<code>login/page.tsx</code>	<code>/login</code>	Sign-in screen
<code>signup/page.tsx</code>	<code>/signup</code>	Create account
<code>apply/[jobId]/page.tsx</code>	<code>/apply/123</code>	Apply to one job
<code>applications/page.tsx</code>	<code>/applications</code>	Track your applications
<code>recruiter/page.tsx</code>	<code>/recruiter</code>	Recruiter dashboard
<code>about</code> , <code>contact</code> , <code>forgot-password</code>	<code>/about</code> ...	Supporting pages
<code>api/jobs/route.ts</code>	<code>/api/jobs</code>	(not a page — a data endpoint, see Ch.7)

The square brackets in `[jobId]` mean "this part of the address changes" — `/apply/abc` and `/apply/xyz` both load the same page with a different job id.

The most important file: `layout.tsx`

`layout.tsx` is the **frame** wrapped around *every* page. It is where the navbar at the top and the footer at the bottom are placed once, so they appear on all screens. It also loads the fonts and switches the whole site into dark mode before the page even paints.

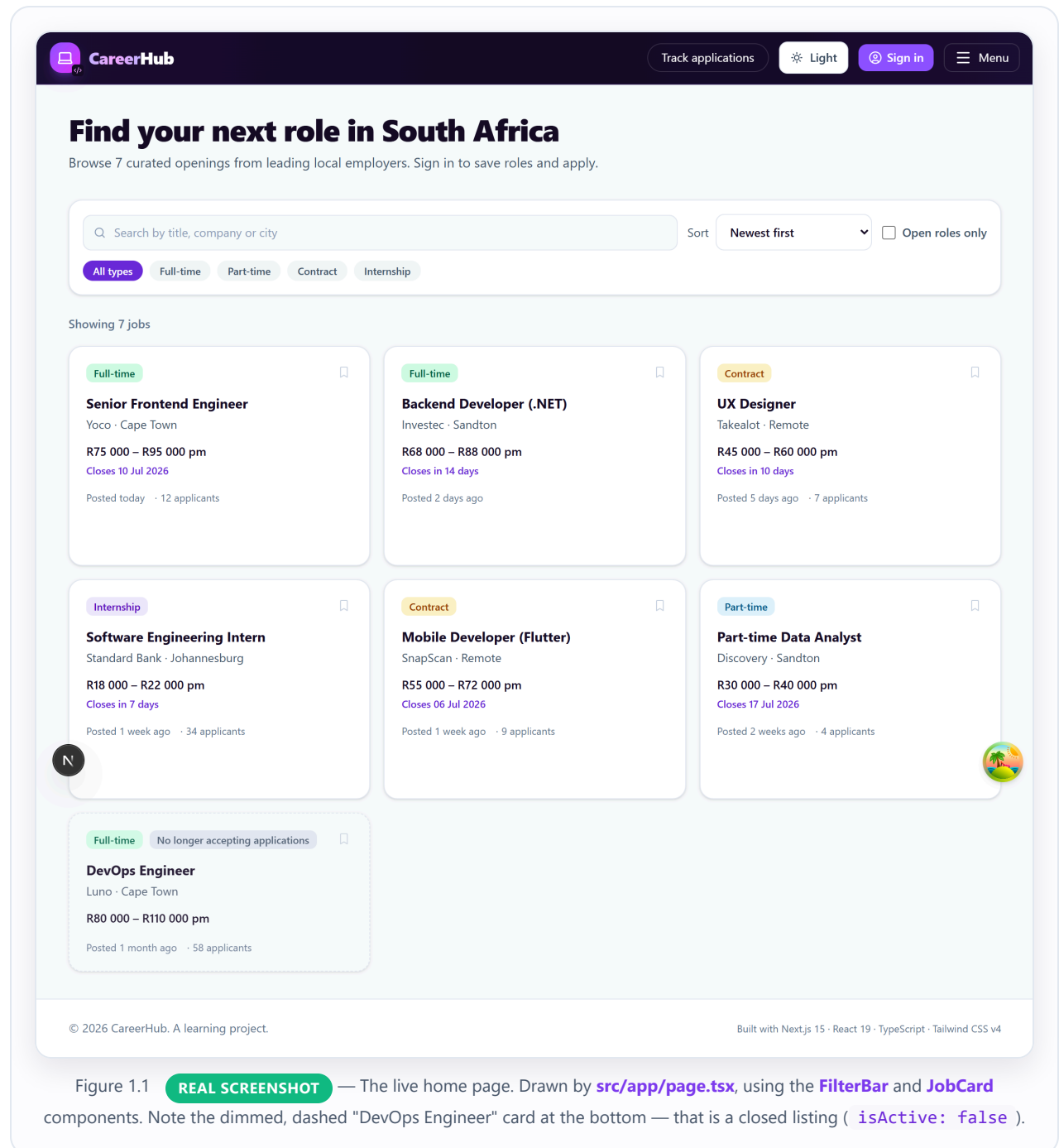
```
// src/app/layout.tsx — the frame every page lives inside
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <Providers>                                {/* gives pages access to server data */}
          <AuthProvider>                            {/* remembers who is logged in (Ch.3) */}
            <ToastProvider>                         {/* enables pop-up messages (Ch.3) */}
            <Navbar />                               {/* the purple top bar (Ch.2) */}
            <main>{children}</main>                  {/* <-- the current page slots in here */}
            <Footer />                               {/* the bottom bar */}
          </ToastProvider>
        </AuthProvider>
      </Providers>
    </body>
  </html>
)
```

```
);  
}
```

`{children}` is the slot where each individual page (home, login, apply...) gets dropped in. Everything around it stays constant.

What the home page draws

The home page (`src/app/page.tsx`) asks for the list of jobs, lets the user search/filter/sort them, and shows them as a grid of cards. Here is a picture of the result:



The same page in dark mode

One click of the theme toggle re-colours the entire interface. Nothing in the page code changes — the styling reacts to a single `.dark` class on the page (see Chapter 2's `ThemeToggle`).

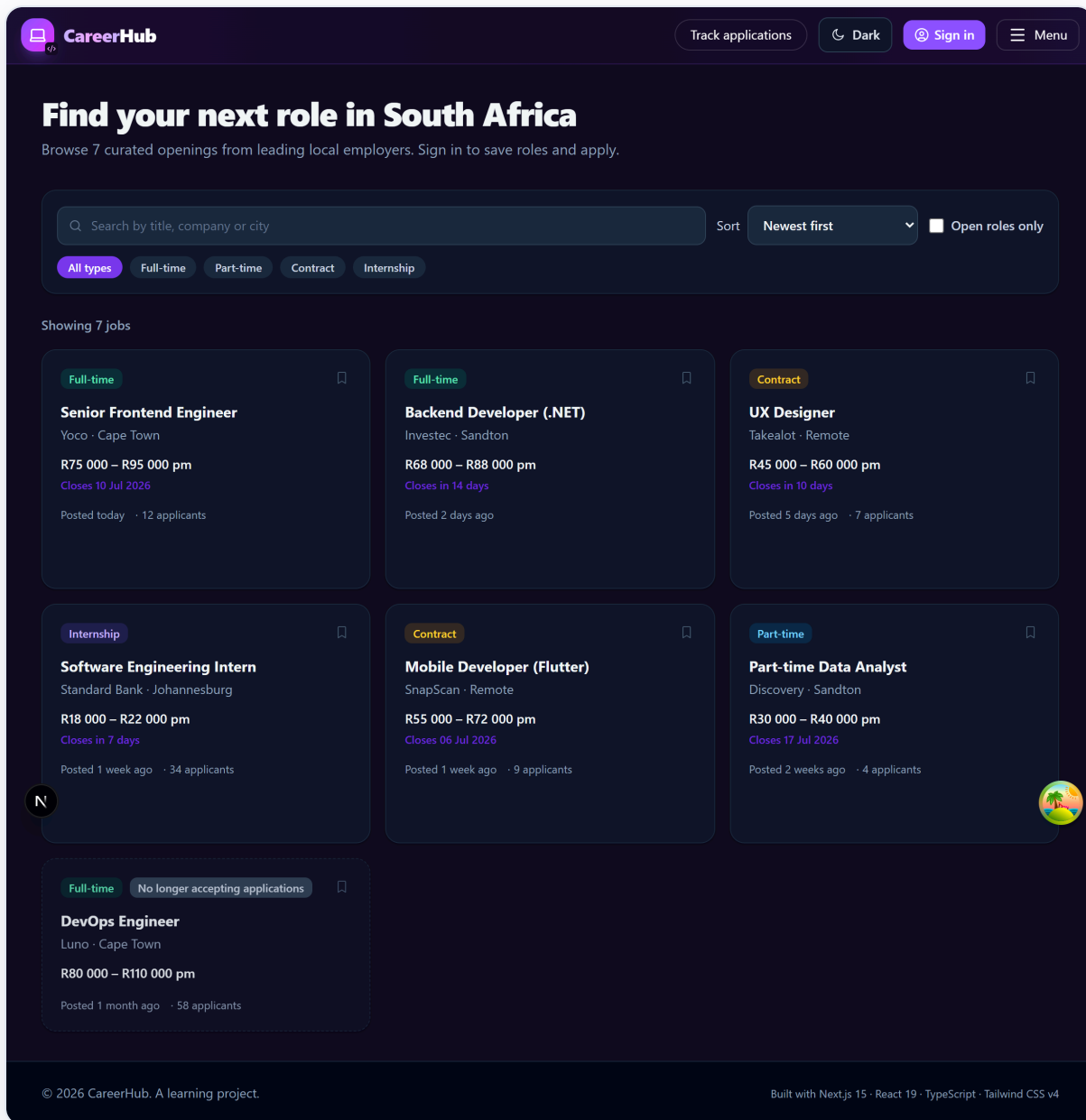


Figure 1.2 **REAL SCREENSHOT** — The identical page, dark theme. Same `page.tsx`, same components — only the colours respond.

The three states of the page (a key beginner idea)

When the page asks for jobs, one of three things is true at any moment. Good UI shows the right thing for each:

```
// src/app/page.tsx — pick exactly one of three views
{isPending ? <JobListSkeleton /> // 1. LOADING — grey placeholder cards
: isError ? <ErrorBox onRetry={refetch} /> // 2. FAILED — message + "Try again"
: <JobList jobs={visibleJobs} /> // 3. SUCCESS — the real cards}
```

Responsible code: the loading skeleton is `components/JobCardSkeleton.tsx` ; the data request lives in `lib/api.ts` (Chapter 5); the cards are `components/JobList.tsx` + `JobCard.tsx` (Chapter 2).

Clicking a card opens the summary panel

When the user selects a job, `page.tsx` records its id in state and renders a detail panel above the grid — full description, responsibilities, qualifications and skills. Click the same card again to deselect and the panel disappears from the page entirely.



Find your next role in South Africa

Browse 7 curated openings from leading local employers. Sign in to save roles and apply.

Sort

Newest first



Open roles only

All types

Full-time

Part-time

Contract

Internship

Full-time

Senior Frontend Engineer

Yoco · Cape Town

SALARY
R75 000 – R95 000 pmPOSTED
todayCLOSING
Closes 10 Jul 2026EXPERIENCE
5+ yearsAPPLICANTS
12

Apply now



ABOUT THE ROLE

Join the merchant dashboard team building the tools that thousands of South African small businesses use every day to track sales, payouts and customers. You'll own end-to-end features from design hand-off to production.

WHAT YOU'LL BE DOING

- Build and ship React features in the merchant dashboard
- Collaborate with designers and backend engineers on new product flows
- Mentor mid-level engineers in code review and pairing sessions
- Improve performance, accessibility and test coverage of existing screens

MINIMUM REQUIREMENTS

QUALIFICATION
Bachelor's degree in Computer Science, Engineering or a related field

EXPERIENCE
5+ years

TOOLS & SKILLS

TypeScript

React

Next.js

Tailwind CSS

Jest

REST APIs



Showing 7 jobs

Full-time

Senior Frontend Engineer

Yoco · Cape Town

R75 000 – R95 000 pm
Closes 10 Jul 2026

Posted today · 12 applicants

Apply now

Full-time

Backend Developer (.NET)

Investec · Sandton

R68 000 – R88 000 pm
Closes in 14 days

Posted 2 days ago

Contract

UX Designer

Takealot · Remote

R45 000 – R60 000 pm
Closes in 10 days

Posted 5 days ago · 7 applicants

Internship

Software Engineering Intern

Standard Bank · Johannesburg

R18 000 – R22 000 pm
Closes in 7 days

Posted 1 week ago · 34 applicants

Contract

Mobile Developer (Flutter)

SnapScan · Remote

R55 000 – R72 000 pm
Closes 06 Jul 2026

Posted 1 week ago · 9 applicants

Part-time

Part-time Data Analyst

Discovery · Sandton

R30 000 – R40 000 pm
Closes 17 Jul 2026

Posted 2 weeks ago · 4 applicants

Full-time

No longer accepting applications

DevOps Engineer

Luno · Cape Town

R80 000 – R110 000 pm

Posted 1 month ago · 58 applicants

Figure 1.3 **REAL SCREENSHOT** — A selected card (highlighted purple) and the **SummaryPanel** it opens. The panel only exists in the page while a job is selected.

State: the page's short-term memory

"State" is data that can change while you use the page, causing the screen to redraw. On the home page, the search text, the chosen filter, and which card is selected are all state:

```
const [query, setQuery] = useState(""); // what's typed in search
const [selectedId, setSelectedId] = useState(null); // which card is clicked
```

When you type, `setQuery` updates the memory and React redraws the list to match — instantly, with no page reload. That responsiveness is the whole point of a modern front-end.

components — reusable screen blocks

A **component** is a self-contained piece of UI you can use again and again, like a LEGO brick. Instead of copy-pasting a job card eight times, you write `JobCard` once and reuse it for every job. Change the brick once, every copy updates.

What lives here

Component file	What it draws on screen
<code>Navbar.tsx</code>	The purple top bar: logo, links, sign-in, menu
<code>Footer.tsx</code>	The bottom bar
<code>JobCard.tsx</code>	One job tile (title, salary, badges, Apply)
<code>JobList.tsx</code>	Arranges many <code>JobCard</code> s into a grid
<code>JobCardSkeleton.tsx</code>	Grey "loading" placeholders
<code>FilterBar.tsx</code>	Search box, sort menu, type filters
<code>SummaryPanel.tsx</code>	Detail panel for the selected job
<code>JobStatusBadge.tsx</code>	The coloured "Full-time" / "Closed" tags
<code>ApplyModal.tsx</code>	The pop-up application form
<code>AuthShell.tsx</code> , <code>AuthFields.tsx</code>	Shared layout & inputs for login/signup
<code>ThemeToggle.tsx</code>	The light/dark switch button
<code>ui/badge.tsx</code>	A tiny generic "pill" used by the badges

Picture: the Navbar



Figure 2.1 **REAL SCREENSHOT** — Drawn by `components/Navbar.tsx`. The "Sign in" button becomes "Sign out" once you log in — it reads the logged-in user from Context (Chapter 3).

```
// src/components/Navbar.tsx — the button changes based on login state
const { user, signOut } = useAuth(); // ask the shared memory: who's logged in?
{user
  ? <button onClick={signOut}>Sign out</button> // logged in -> show Sign out
  : <Link href="/login">Sign in</Link>} // logged out -> show Sign in
```

Props: how a component is told what to show

Props are the inputs you hand a component — like the ingredients you give a recipe. `JobCard` doesn't know about any specific job; you *pass* it one. The same brick draws "Frontend Engineer" or "Data Analyst" depending only on the props it receives.

```
// src/components/JobCard.tsx — its inputs (props) are clearly listed
interface JobCardProps {
  job: JobListing;           // the job to display
  isSelected: boolean;       // draw the purple highlight?
  onSelect: (id) => void;     // what to do when clicked
  isSaved?: boolean;         // is it bookmarked? (the ? = optional)
  onApply?: (id) => void;     // what to do when "Apply" is pressed
}
```

Picture: one Job Card, and the code behind each part

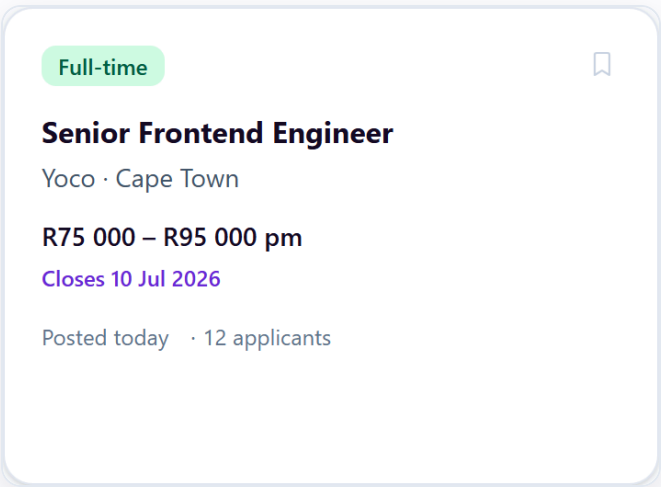


Figure 2.2 **REAL SCREENSHOT** — One live card from `JobCard.tsx`. The card does no formatting itself — it borrows helpers from `lib`.

Part of the card you see	Responsible code
"Full-time" coloured badge	<code>components/JobStatusBadge.tsx</code> + <code>lib/employmentType.ts</code>
Bookmark icon (top-right)	<code>JobCard.tsx</code> via the <code>onToggleSave</code> prop
"R75 000 – R95 000 pm"	<code>lib/format.ts</code> → <code>formatSalaryRange()</code>
"Closes 10 Jul 2026"	<code>lib/format.ts</code> → <code>formatClosingDate()</code>
"Posted today · 12 applicants"	<code>lib/format.ts</code> → <code>formatRelativeDate()</code>
"Apply now" button (on hover)	<code>JobCard.tsx</code> via the <code>onApply</code> prop → <code>/apply</code> page

Beginner takeaway: a component's job is to *arrange and display*. It delegates "how to format money" or "how to colour a badge" to small specialists. That separation is why the code stays clean as the app grows.

context — shared memory across the whole app

Some information is needed in many places at once: "Is anyone logged in?" is needed by the navbar, the home page, the apply button... Passing it by hand through every layer would be painful. **Context** is React's way to put a value in one central place that any component can read directly.

Two contexts in CareerHub

File	What it shares with everyone
<code>context/AuthContext.tsx</code>	The logged-in user, and the <code>signIn</code> , <code>signUp</code> , <code>signOut</code> actions
<code>context/ToastContext.tsx</code>	The <code>notify(...)</code> action that makes a pop-up message appear anywhere

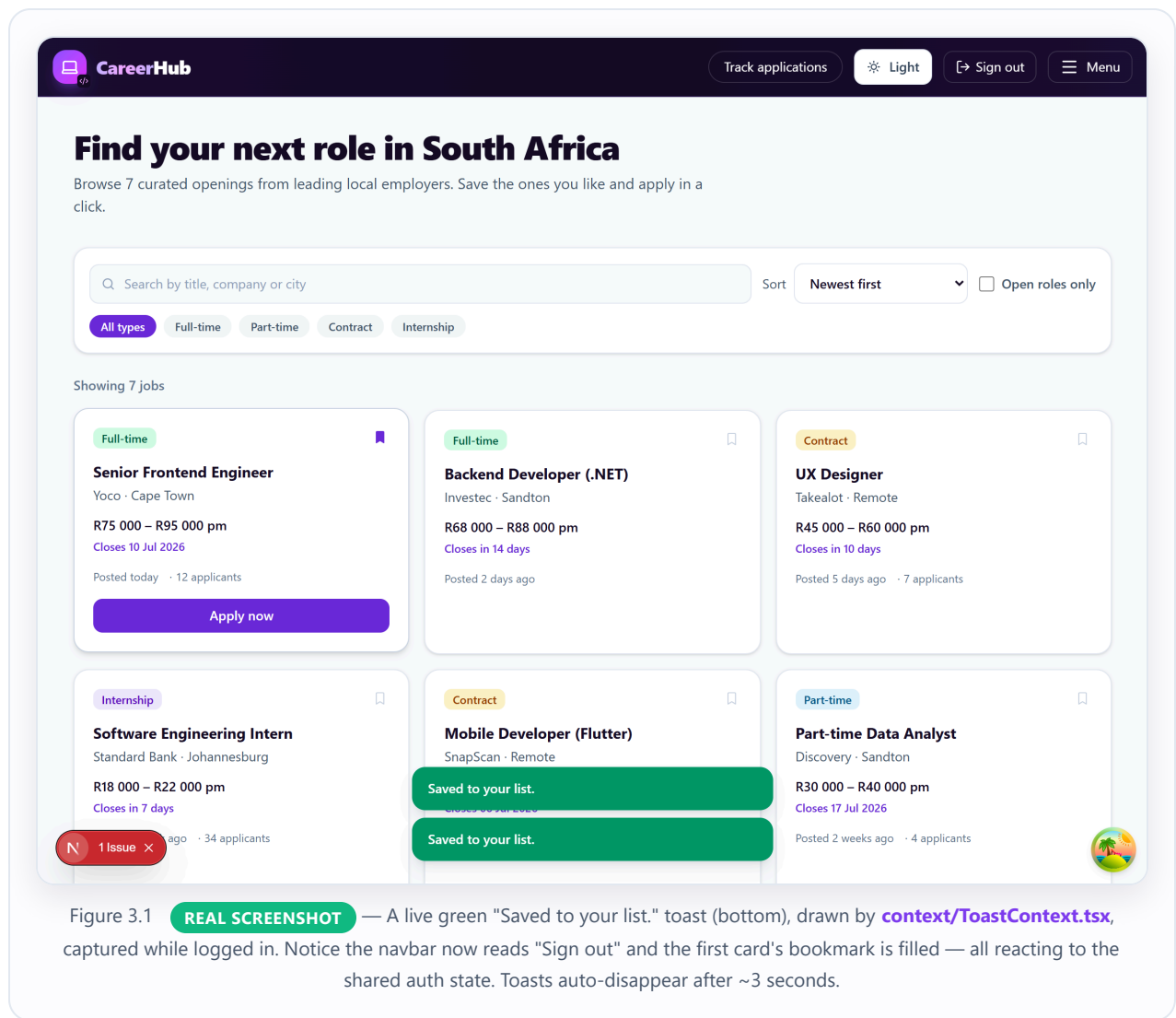
How any component "tunes in"

A component calls a tiny hook — `useAuth()` or `useToast()` — and gets the shared value instantly, no matter how deep it sits in the tree.

```
// Anywhere in the app:
const { user } = useAuth();    // read who is logged in
const { notify } = useToast(); // borrow the pop-up function

notify("Saved to your list.", "success"); // green toast slides up
```

Picture: a toast (pop-up message)



The login screen that feeds the auth context

The `/login` page collects the email and password and calls `signIn` from `useAuth()`. On success the whole app knows a user is present — the navbar, the save and apply buttons all update at once.

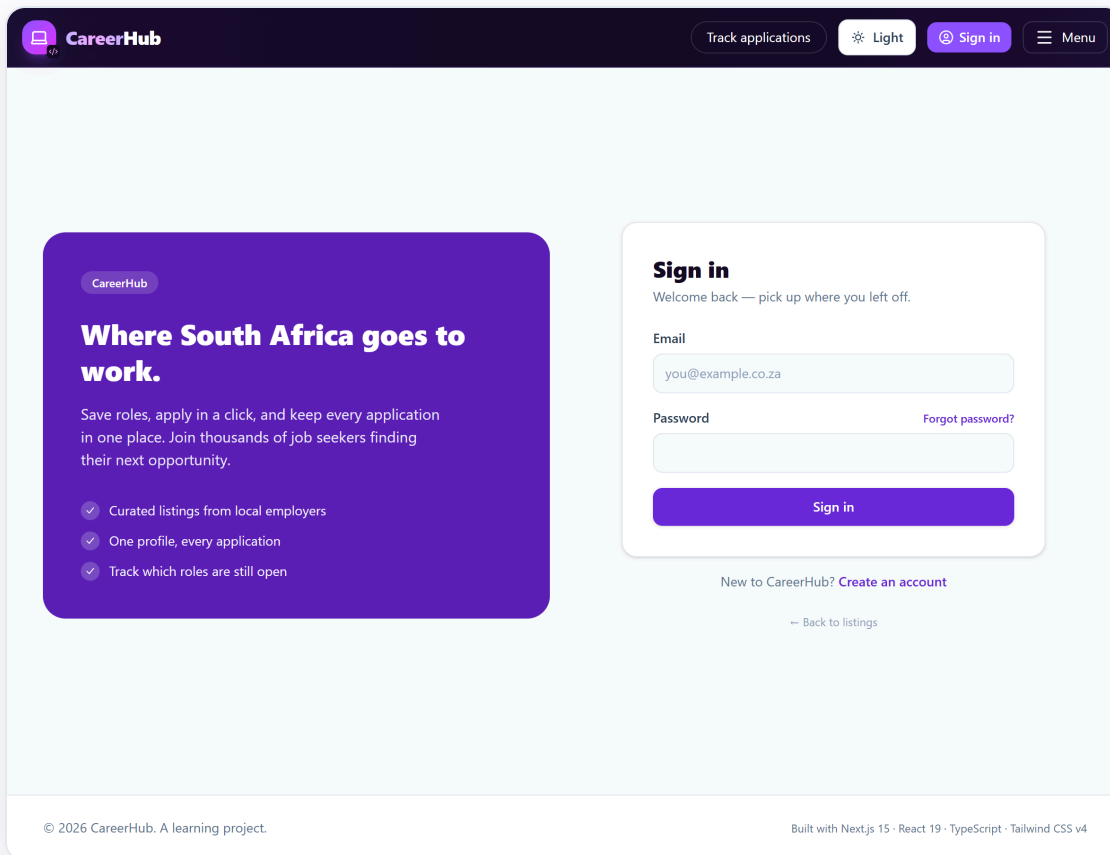


Figure 3.2 **REAL SCREENSHOT** — The live `/login` page (`app/login/page.tsx`), built from the shared `AuthShell` and `AuthFields` components, wired to `AuthContext`.

The "data-source-agnostic" idea (important & clever)

Right now there is no real login server, so accounts are stored in the browser's own storage (`localStorage`). But notice the **shape** of what `AuthContext` offers never changes — `user` , `signIn` , `signOut` . When a real C# login server is built later, **only the inside of these functions changes**; every screen that uses `useAuth()` stays exactly the same.

```
// src/context/AuthContext.tsx — today it reads the browser; tomorrow a server.
const signIn = async (email, password) => {
  // TODAY: look the account up in localStorage
  const account = readAccounts().find(a => a.email === email);
  if (!account || account.password !== password)
    throw new Error("Email or password is incorrect.");
  persistSession(account);
  // LATER: replace the two lines above with: await fetch("/api/login", ...)
};
```

Why this matters: the UI was built to be "swap-ready". The front-end team could finish the entire interface before the C# back-end existed. Chapter 7 completes that story.

fonts — the typefaces

A **font** is the design of the letters. CareerHub ships its own font files so the text always looks the same and loads fast — even with no internet — instead of fetching them from Google's servers on every visit.

What's in the folder

File	Used for
<code>inter-variable.woff2</code>	Inter — the body text (paragraphs, labels)
<code>plus-jakarta-sans-variable.woff2</code>	Plus Jakarta Sans — the bold display text (headings, the logo)

`.woff2` is a highly-compressed web font format — small files, quick downloads. "Variable" means one file contains every weight from thin to extra-bold.

How they get switched on

The fonts are loaded in `app/layout.tsx` (Chapter 1) and attached to the page so the rest of the styling can use them by name:

```
// src/app/layout.tsx
const inter = localFont({ src: "../fonts/inter-variable.woff2", variable: "--font-inter" });
const jakarta = localFont({ src: "../fonts/plus-jakarta-sans-variable.woff2", variable: "--font-jakarta" });

<body className={` ${inter.variable} ${jakarta.variable}`} > ...
```

Picture: the two typefaces in use

CareerHub — Find your next role

↑ Plus Jakarta Sans (display) — headings & logo

Browse curated job listings across South Africa. Save the ones you like and apply in a click.

↑ Inter (body) — paragraphs & labels

Figure 4.1 — Two fonts, two jobs. Headings use the strong display font; body text uses the highly-readable Inter. Both are loaded by `app/layout.tsx`.

Why self-host? Privacy (no request to Google on each visit), performance (no extra connection to a third-party server), and reliability (the app's text looks right even offline). The comment in `layout.tsx` states exactly this reasoning.

lib — the engine room

lib (short for **library**) holds the logic that is *not* a visible component: fetching data, formatting numbers and dates, storing things, and small shared helpers. Keeping it separate means this logic can be tested and reused by any component.

What lives here

File	Job
<code>api.ts</code>	Fetches the job list over the network (the door to the back-end)
<code>seed-jobs.ts</code>	The built-in demo job data
<code>careerhub-store.ts</code>	Saving/reading applications & recruiter jobs in the browser
<code>format.ts</code>	Turning raw values into human text (money, dates)
<code>employmentType.ts</code>	The one place that maps <code>"FullTime"</code> → <code>"Full-time"</code>
<code>utils.ts</code>	<code>cn()</code> — safely combines styling classes

1) Formatting — making data human

The back-end sends a plain number like `45000` and a machine date like `2026-06-17T08:00:00Z`. Nobody wants to read those. `format.ts` converts them into friendly South-African text:

```
// src/lib/format.ts
formatSalaryRange(45000, 65000) // → "R45 000 - R65 000 pm"
formatRelativeDate("2026-06-17...") // → "2 days ago"
formatClosingDate("2026-06-28...") // → "Closes in 9 days"
```

Responsible for what you saw in Figure 2.2: the salary line, the "posted X ago" line, and the "Closes in..." hint all come from this one file.

2) The data door — `api.ts`

This is the only file that actually talks to the server. It asks for jobs and carefully checks the answer succeeded before handing the data back:

```
// src/lib/api.ts
export async function fetchJobs() {
  const res = await fetch(`${baseUrl}/api/jobs`); // ask the server
  if (!res.ok) // 404 / 500? treat as a real error
    throw new Error(`Failed to load jobs - server responded with ${res.status}`);
  return res.json(); // success → give back the jobs
}
```

This is the exact seam where the C# back-end plugs in — explained fully in Chapter 7.

3) Browser storage — `careerhub-store.ts`

Until the real back-end exists, things like submitted applications are saved in the browser so the app feels complete. The functions are named like a database would be — `saveApplication` , `getApplicationsByEmail` — so they can later be swapped for real server calls without changing the pages that use them.

4) `cn()` — tidy styling

CareerHub styles things with Tailwind utility classes. When two classes conflict (e.g. two different borders), `cn()` makes sure the *last* one wins, so the selected-card highlight always overrides the default border correctly.

```
// src/lib/utils.ts
cn("border-slate-200", isSelected && "border-brand-600")
// when selected → keeps only "border-brand-600" (no clash)
```

Beginner takeaway: `lib` is where "how things work" lives, while `components` is "how things look". Splitting them is what keeps each file small and focused.

types — the data contract

CareerHub is written in **TypeScript**, which lets you describe the exact *shape* data must have. The `types` folder holds those descriptions. They are the written agreement — the **contract** — between the front-end and the C# back-end about what a "job" or a "user" contains.

The single file: `types/index.ts`

It defines what every job must include. If any code tries to use a job without, say, a `salaryMax`, or misspells a field, the app **refuses to build** — the mistake is caught on the developer's screen, long before a user ever sees it.

```
// src/types/index.ts — the shape every job must follow
export interface JobListing {
  id: string;
  title: string;
  company: string;
  location: string;
  employmentType: EmploymentType; // must be one of the four below
  salaryMin: number;
  salaryMax: number;
  postedAt: string;                // machine date e.g. "2026-06-17T08:00:00Z"
  isActive: boolean;              // still accepting applications?
  applicantCount: number;
  description: string;
  responsibilities: string[];
  skills: string[];
  closingDate: string;
}

// Only these four words are allowed — nothing else compiles:
export type EmploymentType = "FullTime" | "PartTime" | "Contract" | "Internship";
```

Why a contract prevents bugs

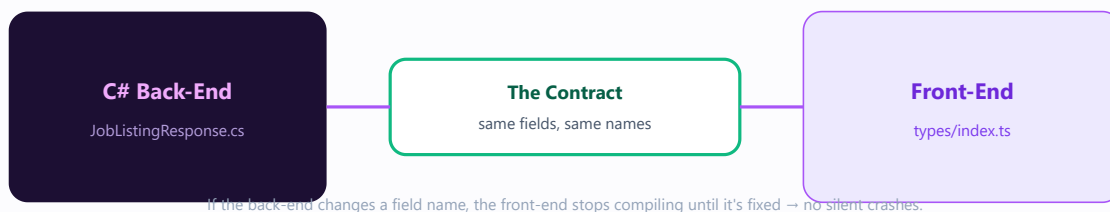


Figure 6.1 — `types/index.ts` mirrors the back-end's `JobListingResponse.cs`. Keeping both in step is what makes the connection safe.

It also defines the `User` shape (id, name, email) used by the login system in Chapter 3. Types draw nothing on screen themselves — but they guard *everything* that does.

Beginner takeaway: types are like the slots in a vending machine. A R5 coin won't fit the R2 slot. TypeScript rejects the wrong shape of data before it can cause a real-world failure.

Front-End connecting to a C# Back-End

So far the front-end has used demo data. A real app needs a **back-end**: a separate program running on a server that owns the real database. CareerHub's back-end is written in **C#** (using ASP.NET). This chapter explains, in plain terms, how two programs written in two different languages talk to each other.

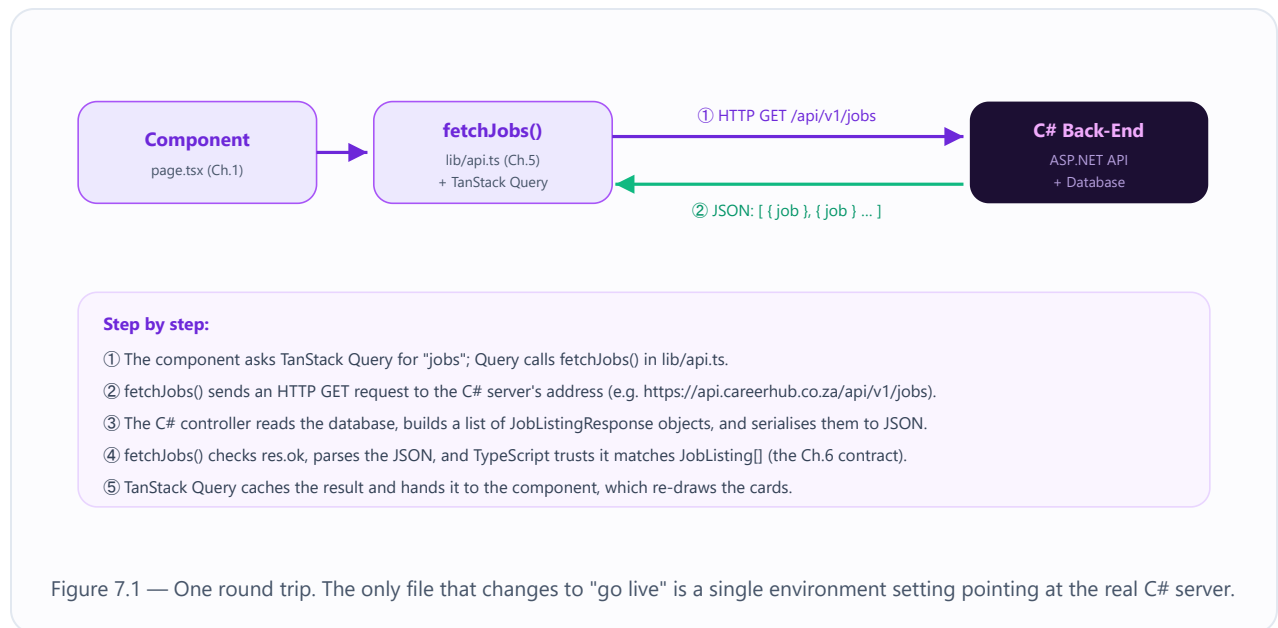
They don't share code — they share a language called HTTP + JSON

The front-end (TypeScript, in the browser) and the back-end (C#, on a server) never call each other's functions directly. Instead they exchange messages over the web using:

- **HTTP** — the same protocol your browser uses to load any website. The front-end sends a *request* ("please give me the jobs"); the back-end sends a *response*.
- **JSON** — a simple, universal text format for data. Both C# and TypeScript can read and write it. It looks like `{ "title": "Frontend Engineer", "salaryMin": 45000 }`.

Because JSON is just text, it does not matter that one side is C# and the other is TypeScript. JSON is the neutral middle-ground both understand — like two people who speak different languages agreeing to write to each other in English.

The full journey of one request



What the C# side roughly looks like

You don't need to write C#, but seeing it demystifies the connection. A C# controller exposes the same `/api/jobs` address the front-end already calls:

```
// C# (ASP.NET) — the back-end half. Returns JSON the front-end already expects.
[ApiController]
[Route("api/v1/jobs")]
public class JobsController : ControllerBase {
    [HttpGet]
    public ActionResult<List<JobListingResponse>> GetJobs() {
        var jobs = _db.Jobs.ToList(); // read the real database
    }
}
```

```
    return Ok(jobs); // ASP.NET converts this to JSON automatically
  }
}
```

The C# class `JobListingResponse` has the same field names as the front-end's `JobListing` interface (Chapter 6) — that shared shape is the whole secret.

The one change to switch from demo to real

Today the front-end points at its own built-in mock route (`src/app/api/jobs/route.ts`). Going live means pointing `fetchJobs()` at the C# server instead — a single address in an environment file:

```
# .env file – change one line to go live
NEXT_PUBLIC_API_URL=https://api.careerhub.co.za # the C# server's address
```

```
// lib/api.ts already reads that setting – no other code changes:
const baseUrl = process.env.NEXT_PUBLIC_API_URL;
const res = await fetch(`${baseUrl}/api/jobs`);
```

Two real-world details the back-end must handle

- **CORS** — browsers block a website from calling a *different* server unless that server says "I allow this site". The C# back-end must enable CORS for the front-end's address.
- **Authentication tokens** — once login is real, the C# server gives the browser a token after sign-in; the front-end then attaches that token to each request so the server knows who is asking. This slots straight into `AuthContext` (Chapter 3) and `api.ts` .

The big picture: the front-end was deliberately built so the back-end's language is irrelevant. C#, Java, Python or Node — as long as it speaks HTTP and returns JSON in the agreed shape, CareerHub's UI works unchanged.

Part 1 Summary — the front-end folders (Chapters 0–6)

The `src` folder is the entire front-end, split into six rooms that each own one clear responsibility. Together they form a clean, swap-ready user interface.

Room	Responsibility	If you remember one thing...
<code>src</code>	The container that organises everything	Structure, not code
<code>app</code>	Pages & URLs (folders become web addresses)	<code>layout.tsx</code> frames every page
<code>components</code>	Reusable visual blocks fed by "props"	How things <i>look</i>
<code>context</code>	Shared memory: login & toasts	<code>useAuth()</code> , <code>useToast()</code>
<code>fonts</code>	Self-hosted typefaces	Fast, private, offline-safe text
<code>lib</code>	Logic: fetching, formatting, storing	How things <i>work</i>
<code>types</code>	The data contract guarding everything	Bugs caught before users see them

The golden thread. Three ideas repeat across every chapter:

- **Separation of concerns** — looks (`components`) are kept apart from logic (`lib`), data shape (`types`) and shared state (`context`).
- **Reuse** — write `JobCard` once, draw it eight times; define a colour or a label in exactly one place.
- **Swap-readiness** — the demo data and mock login can be replaced by a real server with almost no change to the screens, because everything talks through small, well-shaped seams.

A user clicks a card → the component reads it (Ch.1) → draws it with reusable bricks (Ch.2) → formatting comes from `lib` (Ch.5) → the shape is guaranteed by `types` (Ch.6) → login state comes from `context` (Ch.3) → in the chosen font (Ch.4). Every room played its part.

Part 2 Summary — connecting to the C# back-end (Chapter 7)

The front-end and the C# back-end are two separate programs that never share code. They cooperate by exchanging text messages over the web.

- **HTTP** carries the request and response — the same web protocol your browser already uses.
- **JSON** is the shared text format both C# and TypeScript can read and write, making the two languages irrelevant to each other.
- **The contract** (`types/index.ts` ↔ C# `JobListingResponse.cs`) guarantees both sides agree on field names and shapes.
- **One seam** — `lib/api.ts` 's `fetchJobs()` — is the single door to the server. Going live means changing one address (`NEXT_PUBLIC_API_URL`), not the UI.
- **The back-end's housekeeping** — enabling CORS so the browser is allowed to call it, and issuing auth tokens after login — completes a secure, production-ready connection.

Why it was built this way. Because the front-end speaks only HTTP + JSON through one small door, the entire user interface could be finished — and demonstrated with mock data — before the C# back-end was ready. When the server lands, the UI flips from demo to live with a one-line change. That is the pay-off of all the separation described in Part 1.

In one sentence: the front-end *asks* over HTTP, the C# back-end *answers* in JSON shaped to the agreed contract, and a single environment setting decides whether that answer comes from demo data or the real server.

Interview Questions & Answers

Five long questions with model answers for each part of this guide. Written to be understood by a beginner while still impressing an interviewer.

Part 0 — The `src` folder

Q Why is front-end code organised into a `src` folder with sub-folders instead of a few big files?

A Organisation manages complexity. `src` ("source") is the container for all hand-written code, and splitting it into focused sub-folders keeps related things together and unrelated things apart. This makes the code **discoverable** (a page is in `app`, a button in `components`), **safer to change** (editing a formatter can't accidentally break a page), and **easier for teams** (people work in different folders without colliding). As the app grows, this structure is what stops it becoming an unmaintainable tangle.

Q What is the difference between `src` and the built website the user sees?

A `src` is the *source* — the human-written instructions. It is not what runs in the browser directly. A build tool (Next.js) reads `src` and *compiles* it into optimised HTML, CSS and JavaScript that the browser actually downloads. So `src` is the recipe; the running website is the finished meal. Editing `src` and rebuilding is how you change the meal.

Q Someone asks you to change the colour of the salary text on a card. How does the folder structure help you find it?

A The structure tells me where to look. The salary *value* is produced in `lib/format.ts` (`formatSalaryRange`); the salary *styling* is in the component that renders it, `components/JobCard.tsx`. I would not need to search the whole app — the separation of "how data is formatted" (`lib`) from "how it's displayed" (`components`) points me straight to the two relevant files.

Q What does "separation of concerns" mean, and where do you see it across `src` ?

A It means each part of the code has one clear job and doesn't mix responsibilities. In CareerHub: `components` handle appearance, `lib` handles logic and data, `types` defines the data shape, `context` holds shared state, and `app` defines pages/routes. Because these concerns are separated, any one can be changed or tested without disturbing the others — for example, the mock data layer can be replaced with a real API without touching a single component.

Q A teammate wants to add a new "Messages" feature. Walking through the folders, where would each piece go?

A A new page/route → a `messages/` folder in `app` with a `page.tsx`. Visual pieces like a message bubble → `components`. The network/storage logic for messages → `lib`. The shape of a "Message" object → `types`. If the unread-count must be visible app-wide (e.g. in the navbar) → a new context in `context`. The existing structure gives every piece an obvious home, which is the sign of a healthy architecture.

Part 1 — The `app` folder

Q Explain "file-based routing" as if to a non-technical person.

A In Next.js, the folders inside `app` are the website's map. If I make a folder called `about` with a `page.tsx` inside, the address `/about` automatically exists and shows that page — I never write a separate list of routes. The structure on disk and the structure of the website are the same thing. It's like a filing cabinet where the drawer labels are also the addresses people use to reach each document.

Q What is `layout.tsx` and why is it important?

A `layout.tsx` is the shared frame wrapped around every page. CareerHub uses it to place the `Navbar` and `Footer` once, load the fonts, set up the shared providers (auth, toasts, data), and apply dark mode before the first paint. Because it surrounds every page via `{children}`, common elements appear everywhere consistently without being repeated in each page — change the navbar once and every screen updates.

Q What does `[jobId]` in `app/apply/[jobId]` mean?

A The square brackets mark a **dynamic segment** — a part of the URL that changes. One file, `apply/[jobId]/page.tsx`, serves every job: `/apply/abc` and `/apply/xyz` both load it, and the page reads the `jobId` from the URL to know which job to show. Without this, I'd need a separate file per job, which is impossible when jobs come from a database.

Q The home page shows loading, error and success differently. Why does that matter and how is it done?

A Fetching data takes time and can fail, so a good UI never leaves the user staring at a blank or frozen screen. `page.tsx` checks the request's status and renders exactly one of three views: a **skeleton** while loading, an **error box with a Retry button** if it failed, or the **real cards** on success. This is handled cleanly because the data request (via TanStack Query) reports `isPending`, `isError` and the data itself, so the component just maps each state to the right UI.

Q What is "state" on the home page, and give concrete examples.

A State is data that can change while the page is open and, when it changes, causes the screen to redraw. On the home page the search text, the selected employment-type filter, the sort order, the "open roles only" checkbox, and which card is selected are all state. When the user types in the search box, `setQuery` updates the state and React instantly re-filters and re-renders the visible jobs — no page reload. That live responsiveness is the essence of a modern single-page front-end.

Part 2 — The `components` folder

Q What is a component, and what's the benefit of using them?

A A component is a self-contained, reusable piece of UI — like a LEGO brick. CareerHub's `JobCard` is written once and reused for every job in the list. The benefits are **reuse** (no copy-paste), **consistency** (every card looks and behaves the same), and **maintainability** (fix or restyle the card in one place and all instances update). Components also make complex screens readable, because each is a small, named building block.

Q What are "props"? Illustrate with `JobCard`.

A Props are the inputs passed into a component — its ingredients. `JobCard` knows nothing about any specific job; the parent passes it one via the `job` prop, plus `isSelected` (whether to highlight), `onSelect` (what to do on click) and optional `onApply` / `onToggleSave` handlers. The same component renders any job depending purely on the props it receives, which is exactly what makes it reusable.

Q The navbar shows "Sign in" or "Sign out" depending on the user. How does the component know?

A It reads shared state from context: `const { user, signOut } = useAuth()`. If `user` exists, it renders a "Sign out" button wired to `signOut`; otherwise it renders a "Sign in" link. The navbar doesn't manage login itself — it simply reflects the central auth state, so the moment the user logs in or out anywhere, the navbar updates automatically.

Q Why is the colour and label of a status badge defined in dedicated files rather than inside `JobCard`?

A To keep a **single source of truth**. The label text lives in `lib/employmentType.ts` and the colours in `components/JobStatusBadge.tsx`, each defined once as an exhaustive map keyed by the employment type. A card never decides what colour "Contract" is — it asks the badge. If the design changes, I edit one map and every badge across the app updates. It also means adding a new employment type forces me to add its label and colour, or the app won't compile — mistakes are caught early.

Q How do components delegate work to the rest of the app? Use `JobCard` as the example.

A `JobCard` focuses on *arranging* the layout and delegates the specialised work: salary and date text come from `lib/format.ts`, badge colours from `JobStatusBadge.tsx`, class-merging from `lib/utils.ts` (`cn`), and actions like applying or saving are passed in as props and handled by the parent page. This keeps the component small and readable — it's a coordinator, not a do-everything file, which is a hallmark of clean component design.

Part 3 — The `context` folder

Q What problem does React Context solve?

A It solves "prop drilling" — having to pass a value down through many layers of components that don't even use it, just to reach a deep one that does. Context puts a value in one central place that any component can read directly via a hook. CareerHub uses it for things needed app-wide: the logged-in user (`AuthContext`) and the toast notifier (`ToastContext`), so the navbar, home page and apply flow all read them without passing props through every level.

Q Walk through what happens when a user clicks "Save" on a job while logged out.

A The handler checks `user` from `useAuth()` . Because it's `null` , it calls `notify("Sign in to save jobs.", "info")` from `useToast()` — a toast pops up — and then redirects to `/login` . Both the auth check and the toast come from context, which is why any component can perform this flow without owning login or notification logic itself.

Q Explain "data-source-agnostic" auth in CareerHub.

A `AuthContext` exposes a fixed public shape — `user` , `signIn` , `signUp` , `signOut` — but today its functions store accounts in the browser's `localStorage` because there's no auth server yet. When a real C# auth API arrives, only the *insides* of those functions change (a `fetch` call replaces the `localStorage` lookup); every screen calling `useAuth()` stays identical. The UI is decoupled from where the data actually comes from — that's "data-source-agnostic".

Q How does the toast system work, and why does it use context?

A `ToastProvider` holds a list of active toasts in state and exposes a single `notify(message, tone)` function. Calling it adds a toast, which renders as a coloured pill at the bottom of the screen and is auto-removed after about 3 seconds. It's a context because *any* component — save buttons, the apply page, forms — needs to trigger a notification, and context lets them all share one notifier and one on-screen toast area instead of each building their own.

Q Why does `useAuth` throw an error if used outside an `AuthProvider` ?

A Because reading auth state only makes sense inside the provider that supplies it. If a component calls `useAuth()` with no `AuthProvider` above it, the context value is `null` and any use of `user` would fail mysteriously later. Throwing a clear error immediately — "useAuth must be used within an AuthProvider" — turns a confusing runtime bug into an obvious, self-explaining one, which is good defensive design.

Part 4 — The `fonts` folder

Q Why does CareerHub ship its own font files instead of loading them from Google Fonts?

A Three reasons stated in the code: **privacy** (no request to a third-party server on every visit, so no user data leaks to Google), **performance** (no extra network connection to set up; the fonts come from the app's own server), and **reliability** (the text renders correctly even offline or if Google is unreachable). Self-hosting trades a slightly larger initial bundle for control and consistency.

Q What is a `.woff2` "variable" font?

A `.woff2` (Web Open Font Format 2) is a heavily compressed font format designed for the web — small files and fast downloads. "Variable" means a single file contains a continuous range of weights (e.g. thin through extra-bold) instead of needing a separate file per weight. CareerHub's Inter file covers weights 100–900 in one download, which is efficient and flexible.

Q There are two fonts. What is each used for and why use two?

A Inter is the **body** font — highly legible for paragraphs and labels. Plus Jakarta Sans is the **display** font — used for headings and the logo where a stronger, more characterful look helps. Pairing a clean body font with a distinctive display font creates visual hierarchy: the eye immediately distinguishes titles from content, which improves readability and branding.

Q How does a font defined in `fonts` actually reach the text on screen?

A `app/layout.tsx` loads each file with Next.js's `localFont`, assigning it a CSS variable (`--font-inter` , `--font-jakarta`). Those variables are attached to the `<body>` . The Tailwind styling then references them (e.g. a `font-display` class for headings), so any element can opt into either typeface. The font files sit in `fonts` ; `layout.tsx` is the wiring that switches them on.

Q A teammate says "fonts are just decoration, low priority." How do you respond?

A Typography is part of usability and brand, not mere decoration. Readable body text reduces strain and helps users scan listings quickly; a consistent display font makes the product feel polished and trustworthy. Self-hosting also has real engineering consequences for privacy and load performance. So while it's a small folder, it affects accessibility, perceived quality and even compliance — worth getting right.

Part 5 — The `lib` folder

Q What kind of code belongs in `lib`, and how does it differ from `components`?

A `lib` holds non-visual logic: network calls (`api.ts`), data storage (`careerhub-store.ts`), formatting (`format.ts`), shared maps (`employmentType.ts`) and helpers (`utils.ts`). `components` handle how things *look*; `lib` handles how things *work*. Keeping logic out of components makes it reusable across many components and testable on its own, without rendering any UI.

Q Why does `fetchJobs` check `res.ok` and throw an error?

A Because `fetch` only rejects on network-level failures (offline, DNS); an HTTP error like 404 or 500 still *resolves* with `res.ok === false`. Without the check, the error response body would be parsed as if it were job data and shown as a fake "success". Throwing on `!res.ok` — with the status code in the message — routes real server errors into the UI's error state, so the user sees a proper "couldn't load jobs, try again" message instead of a broken screen.

Q Explain what `format.ts` does and why it's separate from the components.

A It turns raw machine values into human-friendly South African text: a number becomes `R45 000 - R65 000 pm`, a timestamp becomes `2 days ago` or `Closes in 9 days`. It's separated so the exact same formatting is reused by every card and panel, can be unit-tested in isolation, and changed in one place. If a component formatted money inline, every card would risk doing it slightly differently.

Q What is `cn()` in `utils.ts` and what problem does it solve?

A `cn()` combines CSS class names, dropping falsy ones and — crucially — resolving conflicting Tailwind utilities so the last one wins. For example a card has a default border, but when selected it needs a brand-coloured border; `cn()` ensures only the brand border survives rather than emitting both and letting stylesheet order decide unpredictably. It makes conditional styling reliable and readable.

Q How does `careerhub-store.ts` prepare the app for a future real back-end?

A It wraps all data access in named functions — `saveApplication`, `getApplicationsByEmail`, `getJobById` — that today read and write the browser's `localStorage`. The components call these functions, not storage directly. Later, the bodies can be swapped to call the C# API instead, and because the function names and return shapes stay the same, no component needs to change. It mimics a database interface so the swap is seamless.

Part 6 — The `types` folder

Q What is TypeScript, and what does the `types` folder define?

A TypeScript is JavaScript with type-checking: you describe the shape data must have, and the compiler verifies your code respects it before the app even runs. The `types` folder (`index.ts`) defines the core shapes — `JobListing` (every field a job must have) and `User` — plus the `EmploymentType` union. These are the agreed contracts the rest of the app is checked against.

Q How does a type catch a bug before a user ever sees it?

A If any code uses a job incorrectly — a missing `salaryMax` , a misspelt field, or an employment type outside the allowed four — the project fails to compile and the error appears on the developer's screen immediately. As the comment in the file says, a compile error is the correct failure mode, not a runtime crash. The mistake is stopped at build time rather than becoming a broken page in production.

Q Why is `EmploymentType` a union of four strings instead of just `string` ?

A Because only four values are valid: `"FullTime"` , `"PartTime"` , `"Contract"` , `"Internship"` . Typing it as a union means any other value is rejected by the compiler the moment it's written. It also makes related maps (labels, badge colours) **exhaustive** — add a fifth type to the union and TypeScript forces you to supply its label and colour everywhere, so nothing is forgotten. Plain `string` would allow typos and silent gaps.

Q How do the front-end types relate to the C# back-end?

A They mirror it. `JobListing` is the front-end's copy of the back-end's `JobListingResponse.cs` — same field names and shapes. That shared contract is what lets the two programs exchange JSON safely. If the back-end renames a field, the front-end types are updated, and every place that used the old name fails to compile until it's fixed — turning a potential silent production bug into an obvious build error.

Q Types render nothing on screen. Why are they still essential to the UI?

A Because they guarantee that everything which *does* render is working with valid data. They're like the slots in a vending machine that only accept the right coin: they make sure a job always has a title, a salary and a valid employment type before a card tries to display it. This prevents whole classes of UI crashes ("cannot read property of undefined") and documents, precisely, what data each screen can rely on.

Part 7 — Front-End connecting to the C# back-end

Q Two programs in two different languages (TypeScript and C#) need to work together. How is that possible?

A They never share code or call each other's functions. Instead they exchange messages over **HTTP** (the web's request/response protocol) using **JSON** (a universal text format both languages can read and write). The front-end sends an HTTP request; the C# back-end replies with JSON. Because JSON is just text in an agreed shape, the languages on each side are irrelevant — it's a neutral common ground, like two speakers agreeing to correspond in a shared written language.

Q Trace one request for the job list from click to cards on screen.

A The home page asks TanStack Query for the `"jobs"` data; Query runs `fetchJobs()` in `lib/api.ts`, which sends an HTTP GET to the C# server's `/api/jobs` address. The C# controller queries the database, builds a list of `JobListingResponse` objects and returns them; ASP.NET serialises that to JSON. `fetchJobs()` checks `res.ok`, parses the JSON (trusted to match the `JobListing[]` contract), and Query caches it and hands it to the component, which re-renders the cards. One clean round trip.

Q What single change moves CareerHub from demo data to the real C# server?

A Pointing `fetchJobs()` at the real server's address. `api.ts` already reads `process.env.NEXT_PUBLIC_API_URL`, so setting that environment variable to the C# server's URL (instead of the local mock route) is essentially the only change. No components, types or pages need editing — the entire UI was built behind that one seam precisely so going live is a configuration change, not a rewrite.

Q What is CORS, and why does the C# back-end need to handle it?

A CORS (Cross-Origin Resource Sharing) is a browser security rule: a web page is blocked from calling a server at a *different* origin unless that server explicitly permits it. Since the front-end (e.g. `careerhub.co.za`) and the API (e.g. `api.careerhub.co.za`) are different origins, the C# back-end must send CORS headers allowing the front-end's address. Without it, the browser would refuse the response even if the server answered correctly.

Q How does login (auth) fit into the front-end/back-end connection?

A Once login is real, the user submits credentials, the C# server verifies them and returns a **token** (a signed proof of identity). The front-end stores it — slotting straight into `AuthContext` — and attaches it to the header of each subsequent request in `api.ts`, so the server knows who is asking and can authorise the action. Because `AuthContext` already exposes a stable `signIn / user` shape, only its internals change from mock to real; the UI consuming it stays the same.

— End of guide —

CareerHub · `src` folder explained for beginners · Chapters 0–7